

Casos de Uso Detalhados — Chatbot de Música

Integrações: Spotify Web API oficial e ReccoBeats

Canais: Web e Telegram

Perfis: Usuário e Admin

Idiomas: PT-BR e EN-US

Escopo: leitura de dados e criação de playlists

Grupo: Arthur Fernandes, Murilo Spalenza e Vinícius Machado

UC-01 — Autenticar usuário via OAuth (Spotify)

Ator primário: Usuário

Objetivo: Autorizar o chatbot a criar playlists na conta do usuário.

Pré-condições: Aplicação registrada no Spotify (client_id/client_secret inexistentes hoje → precisam ser criados).

Pós-condições: Access token e refresh token armazenados com segurança.

Fluxo principal:

1. Usuário solicita criação de playlist.
2. Sistema verifica ausência de token válido.
3. Redireciona para OAuth Authorization Code Flow do Spotify.
4. Usuário concede permissão (`playlist-modify-private` e/ou `playlist-modify-public`).
5. Sistema recebe authorization code.
6. Troca code por access_token + refresh_token.
7. Token persistido de forma criptografada.

Fluxos alternativos:

- 3A: Usuário nega permissão → sistema cancela criação e informa limitação.
- 6A: Token inválido → reinicia fluxo OAuth.

Observações técnicas:

- Necessário implementar refresh automático.
 - Armazenamento deve seguir LGPD e política do Spotify.
-

UC-02 — Consultar música, artista, álbum ou playlist

Ator primário: Usuário

Objetivo: Buscar metadados musicais.

Pré-condições: Nenhuma (uso de client credentials).

Pós-condições: Resposta estruturada com dados relevantes.

Fluxo principal:

1. Usuário envia query (“Me fale sobre o cantor Luan Santana”).
2. Sistema identifica intenção `search_entity`.
3. Extrai entidade (artista/faixa/álbum).
4. Chama endpoint `/search` da Spotify Web API.
5. Retorna nome, popularidade, gêneros, seguidores, etc.

Fluxos alternativos:

- Ambiguidade → solicitar confirmação.
 - Nenhum resultado → sugerir reformulação.
-

UC-03 — Obter Audio Features de uma faixa

Ator primário: Usuário

Objetivo: Analisar características técnicas (BPM, energy, valence etc.).

Fluxo principal:

1. Usuário solicita análise (“Analise a música The Business - Tiësto”).
2. Sistema busca `track_id` no Spotify.
3. Chama endpoint `/audio-features/{id}`.
4. Opcionalmente complementa com dados da ReccoBeats.
5. Retorna: tempo, key, mode, danceability, energy, acousticness, instrumentalness, valence.

Pós-condição: Resposta interpretável (não apenas números brutos).

UC-04 — Gerar recomendação baseada em similaridade

Ator primário: Usuário

Exemplo: “Me recomende uma música parecida com Evidências - Chitãozinho e Xororó”.

Fluxo principal:

1. Identificar faixa base.
2. Recuperar audio features.
3. Chamar endpoint `/recommendations` (Spotify) com `seed_track`.
4. Opcional: complementar via ReccoBeats (similarity endpoints).
5. Rankear por proximidade de features.
6. Retornar lista com justificativa objetiva.

Critério técnico de similaridade (exemplo):

- Distância vetorial sobre (BPM, energy, danceability, valence).
 - Peso maior para energy e tempo em músicas eletrônicas.
-

UC-05 — Gerar playlist temática (semântica contextual)

Ator primário: Usuário

Exemplo: “Faça uma playlist para estudar num dia chuvoso”.

Fluxo principal:

1. Identificar intenção `generate_playlist_theme`.
 2. Mapear termos semânticos → parâmetros musicais:
 - “estudar” → baixo valence, instrumental, BPM moderado.
 - “dia chuvoso” → acousticness alto.
 3. Gerar seeds por gênero ou artista.
 4. Chamar `/recommendations`.
 5. Retornar lista sugerida (10–30 faixas).
-

UC-06 — Criar playlist na conta do usuário

Ator primário: Usuário autenticado

Objetivo: Persistir playlist gerada.

Pré-condição: UC-01 concluído.

Fluxo principal:

1. Usuário aprova criação.
2. Sistema chama `/users/{user_id}/playlists`.
3. Obtém `playlist_id`.
4. Adiciona faixas via `/playlists/{playlist_id}/tracks`.
5. Retorna link da playlist.

Fluxos alternativos:

- Token expirado → refresh automático.
 - Falha na API → retry controlado.
-

UC-07 — Listar colaborações ou participações de artista

Exemplo: “Quero 10 músicas onde Luan Santana aparece”.

Fluxo principal:

1. Buscar artista_id.
 2. Chamar `/search` com filtro `artist:.`
 3. Filtrar onde artista aparece como primary ou featured.
 4. Ordenar por popularidade.
 5. Limitar a 10 resultados.
-

UC-08 — Descobrir artista pouco conhecido similar a outro

Exemplo: “Quero descobrir uma banda pouco conhecida com músicas parecidas com Nickelback”.

Fluxo principal:

1. Buscar artista base.
 2. Chamar endpoint de artistas relacionados (`/artists/{id}/related-artists`).
 3. Filtrar por popularidade abaixo de threshold.
 4. Validar similaridade por gêneros + audio features médios.
 5. Retornar sugestão.
-

UC-09 — Histórico e métricas

Ator: Admin

Objetivo: Auditoria e análise de uso.

Fluxo principal:

1. Registrar todas as queries.

2. Registrar tempo de resposta e sucesso/erro.
 3. Permitir exportação PDF ou texto simples.
 4. Exibir total de requisições por período.
-

UC-10 — Exportar relatório em PDF ou texto

Ator: Usuário ou Admin

Fluxo principal:

1. Selecionar intervalo.
 2. Sistema gera relatório estruturado.
 3. Exporta em PDF ou `.txt`.
-

UC-11 — Suporte multilíngue

Fluxo principal:

1. Detectar idioma.
 2. Ajustar resposta.
 3. Se idioma não suportado → fallback para PT-BR.
-

Requisitos Não Funcionais (Derivados)

1. **Segurança:** criptografia de tokens e dados pessoais.
2. **Rate limit handling:** controle de chamadas à Spotify e ReccoBeats.
3. **Disponibilidade:** fallback textual caso APIs externas estejam indisponíveis.
4. **Latência alvo inicial (estimativa técnica realista):** 1–3 segundos por consulta.
5. **Conformidade:** retenção e exclusão conforme LGPD e política do Spotify.